AGENT FIGHT CLUB #4

CONTEXT MANAGEMENT

ANATOMY · ROT · CACHING · TECHNIQUES

AGENTFIGHTCLUB.TODAY

CLONE THE REPOS PLEASE

DECK & NOTES



https://github.com/maxsoldev/agent-meetup

EDUAGENT HARNESS



https://github.com/maxsoldev/eduagent

AGENDA

01 Introduction round

02 Part 1: Theory of context management

03 Part 2: Build & Compete

INTRODUCTION ROUND

// 01



WHO ARE YOU?

// 02



WHAT DO YOU EXPECT FROM THE EVENT AND COMMUNITY?

LONG-TERM AGENDA

01 Agent Foundations
Agent loop + basic tools

02 TODAY **Context Management**
Anatomy, context rot, caching, compaction

03 More harness features
Skills, MCP, hooks

04 Agent memory & search
RAG, web search

05 Self-improving agents
e.g. Hermes, OpenClaw

06 Agent-to-Agent (A2A)
Agentic commerce

07 AgentOps
Observability, evals

08 Dark factories & the future of work

PART 1

CONTEXT MANAGEMENT

"A lot of apparent **“model quality”** is really context quality"
— Sebastian Raschka

AGENDA FOR TODAY

01 Context Management Overview

1.1 Context Anatomy: what is the context window and what goes in it

1.2 Context Rot: why longer context hurts

1.3 Caching: make-it-or-break-it for prod agents

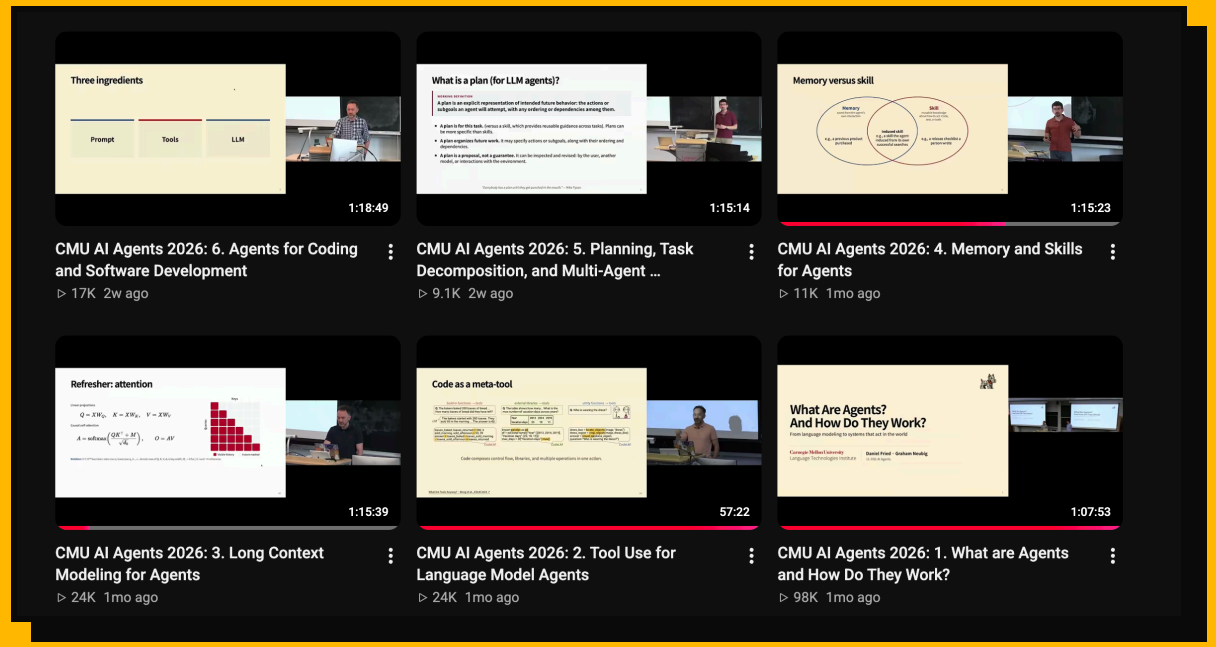
1.4 Context engineering techniques

1.5 So what? Lessons for everyday interactions with agents

1.6 Demo: custom educational harness based on codex

02 Workshop: manage context for trader agents

SUGGESTED SOURCES



CMU 11-768 AI Agents 2026 · Neubig & Fried
youtube.com/playlist?list=PLSN0qpDfUvTM

- **How Claude Code works**
  code.claude.com/docs/en/how-claude-code-works
- **Manus: Context Engineering for AI Agents**
  manus.im/blog
- **Raschka: Components of a Coding Agent**
  magazine.sebastianraschka.com
- **LangChain: Context Engineering for Agents**
  blog.langchain.com/context-engineering-for-agents

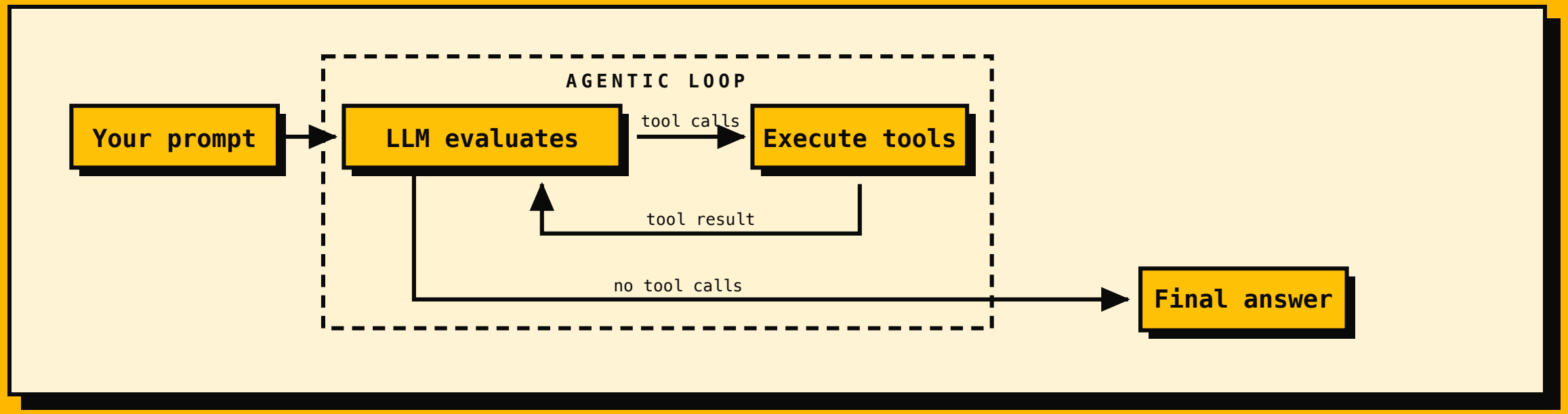
1.1

CONTEXT ANATOMY

What the model actually receives. Spoiler: **far more** than your prompts and tool results.

AGENT LOOP REFRESHER

> An agent is **an LLM autonomously using tools in a loop**
>
> — SIMON WILLISON

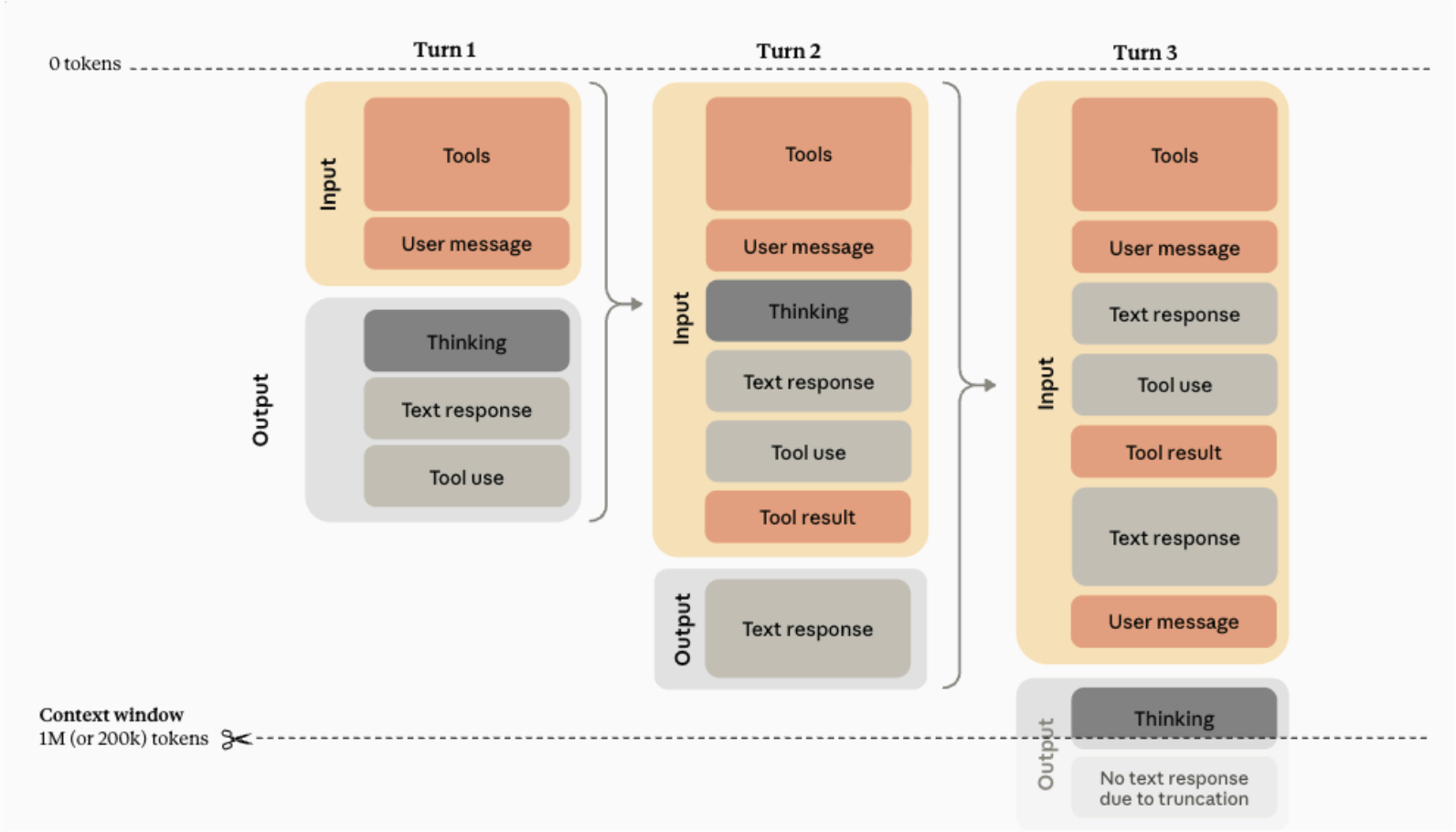


AGENT TURN
One pass through the loop: one model call plus the tool calls it asks for.

AGENT RUN
One user message and all the turns after it, until the model replies with no tool calls.

AGENT SESSION
All the runs that share one message history.

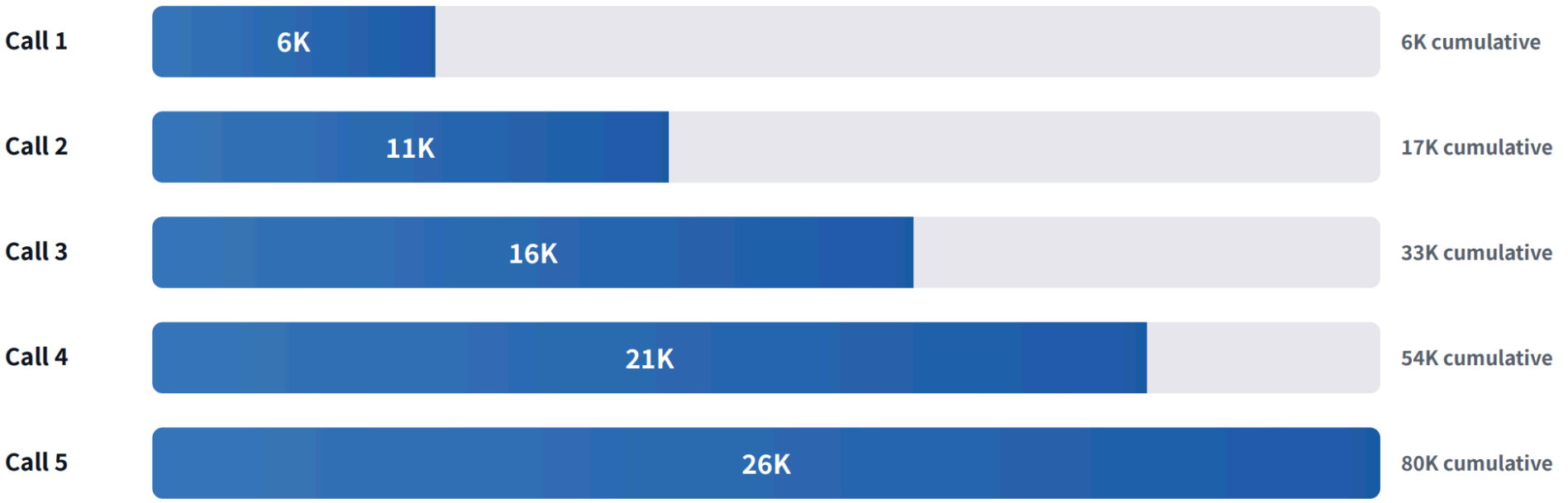
CONTEXT WINDOW



Problem: the context window is finite. What belongs in it? How do we manage long sessions?

RESEND ALL PREVIOUS INPUT

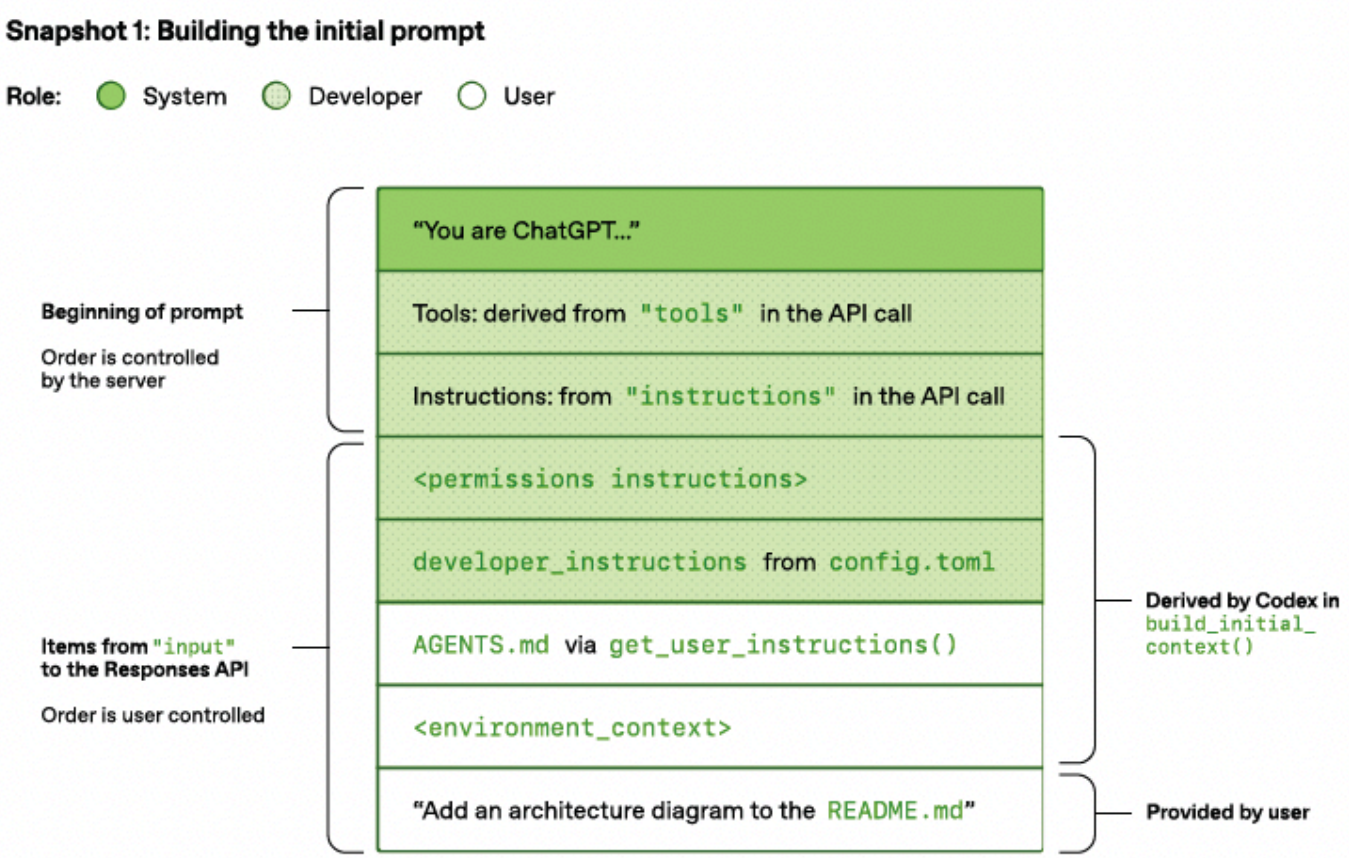
$input_t$ = fixed prefix + history before call t



Each call grows linearly → the total grows quadratically

Every call resends the whole history: 6K + 11K + 16K + 21K + 26K = 80K after 5 calls

INITIAL PROMPT ANATOMY



Your message is a small part of the context. What fills the rest?

FULL CONTEXT ANATOMY

1 Base instructions

- Harness system prompt

2 Tool definitions

- Built-in tools
- MCP tools

3 Developer context

- Permissions
- Developer instructions, mode
- Skills: name + description
- Memories, plugins, apps

4 User context

- AGENTS.md
- Environment: cwd, date, shell

5 Conversation

- User messages
- Assistant messages
- Reasoning items
- Tool calls + outputs

6 Injected mid-session

- Context updates
- Model switch, time, budget
- Hooks, subagents
- Compaction summary
- File-specific rules

7 Hidden by provider

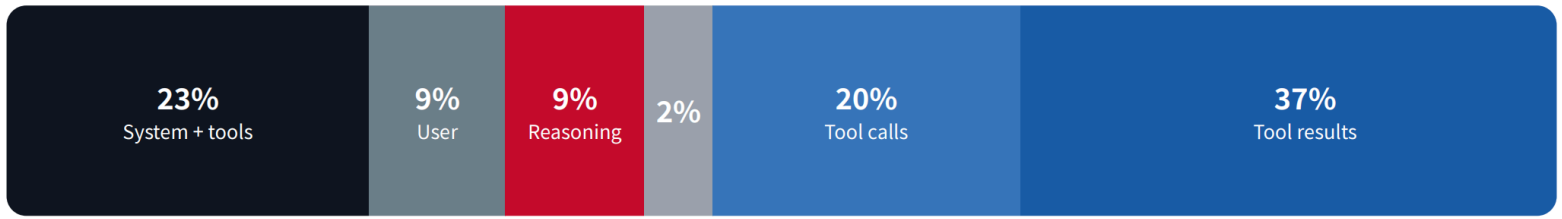
- Chat template tokens
- Tool schemas as text
- Provider system message

8 Output

- Reply
- Reasoning tokens

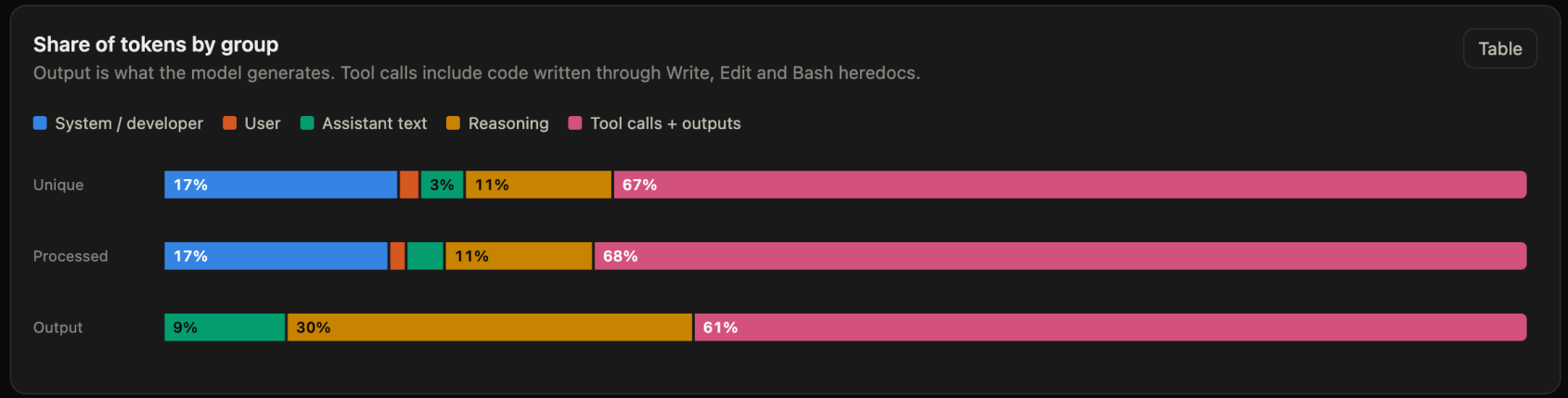
WHERE DO THE TOKENS GO?

OpenHands
1,500 sessions, mean 78K tokens



77,922 mean tokens per session across 1,500 OpenHands sessions

My Claude Code sessions
from ~/.claude/projects



Tool calls + results fill 57–68% of the window. Your prompt is a rounding error.

TIP Clear old tool output. The context gets smaller, and the user–agent conversation stays word for word.

TOKENOMICS: AGENTS PAY TO READ, NOT TO WRITE

TOKEN TYPE · SHARE OF ALL TASK TOKENS



STAGE · AVG SHARE OF TASK TOKENS, WHEN THE STAGE RAN



Code review costs ~7× more tokens than writing the code

ChatDev multi-agent system, GPT-5, 30 tasks · Salim et al., Tokenomics, arXiv 2601.14470

DEMO

EDUAGENT CONTEXT DEMO

An educational harness based on Codex: see **every token** the model receives.

1.2

CONTEXT ROT

Why quality drops and latency grows with **context length**.

As context grows, the model gets **worse at using it**

A model that accepts 1M tokens doesn't use 1M tokens well.
And **every request gets slower, less accurate and more expensive**.

CONTEXT ROT · CHROMA, 2025 · ANTHROPIC, 2025

WHY WE WANT LONG CONTEXT ANYWAY

// 01



LONG TASKS

- Migrations, large refactors
- Long autonomous runs
- Deep research

// 02



BIG INPUTS

- One big document
- Long media
- Many-shot examples

// 03



UNKNOWN SCOPE

- Hard, vague tasks
- You can't pick the context upfront

Some tasks need long context. So we must know what it costs

THE PRICE OF LONG CONTEXT

THE MODEL GETS WORSE

01

WORSE QUALITY

Weaker reasoning, repeated mistakes

02

WORSE ACCURACY

Forgets instructions, mixes up facts

EACH REQUEST GETS SLOWER AND PRICIER

03

SLOWER

Long wait for the first word

04

MORE EXPENSIVE

Bills grow faster than the session

BAD CONTEXT MAKES 01 AND 02 WORSE

Poisoning
A hallucination keeps being referenced

Distraction
The agent repeats past actions

Confusion
Too many tools to choose from

Clash
Parts of the context contradict each other

Drew Breunig, How Long Contexts Fail, 2025

WHY THE MODEL GETS WORSE

// 01



FIXED ATTENTION BUDGET

Attention weights add up to 100%. More tokens = less for each. A highlighter with a fixed amount of ink.

// 02



POSITION BIAS

Models favour the start and the end, whatever is there. The middle gets lost.

// 03



LITTLE PRACTICE

Long documents are rare in training data. Models can **run** at 1M tokens, not **use** them well.

Anthropic, Effective context engineering, 2025 · Liu et al., Lost in the Middle, 2023 · Neubig, CMU 11-768, 2026

CONTEXT ROT IN NUMBERS

Quality
The model gets worse

−14–85%
Scores drop from length alone, even if the extra tokens are whitespace
DU ET AL., 2025

99 → 70%
Retrieval at 32K when question and answer share no words
NOLIMA, 2025

68%
Best models follow only 68% of 500 instructions
IFSCALE, 2025

Cost
100 steps, 20K prompt, +2K per step

10×
Step 100 costs 10× step 1
ILLUSTRATIVE

$36 → $4.4
No cache vs cache. Cheaper, but still quadratic
CLAUDE API PRICES

3×
Fewer tokens: 5 fresh sessions of 20 steps vs one of 100
ILLUSTRATIVE

A long session costs more per step. Caching makes it cheaper, not better at scale

SO WHAT?

01 New task → new session. Compact or hand off with a short summary

02 Put what matters at the start or the end. Restate the goal (todo lists)

03 Correct early or start over. Don't argue with a poisoned context

04 Don't break the cache mid-session: same model, same tools

1.3

CACHING

The rule everything else must respect. **Make-it-or-break-it** for prod agents.

WHAT IS PROMPT CACHING?

WHAT

Save the result of an expensive computation (processing the prompt prefix) and reuse it, instead of computing it again.

HOW

Match the start of the request (the prefix) against recent requests

Hit
Read the prefix from cache: ≤0.1× input price

Miss
Compute it all, write to cache: 1.25× input price

> **"The KV-cache hit rate is the single most important metric for a production-stage AI agent"**
>
> — YICHAO JI, MANUS, 2025

WHY CACHING MATTERS

10×

CHEAPER

Cache reads cost 0.1× the input price (20× cheaper on Opus and Sonnet 5.5). A write costs 1.25×, so it pays off after one read

CLAUDE & OPENAI PRICING

−67%

FASTER

Time to first token: −7% at 1K tokens, −67% at 150K+. The longer the prompt, the bigger the win

OPENAI, PROMPT CACHING 201

5×

MORE THROUGHPUT

Cache reads don't count toward Claude rate limits. 2M tokens/min limit + 80% hits = 10M tokens/min

CLAUDE RATE LIMITS

0

QUALITY COST

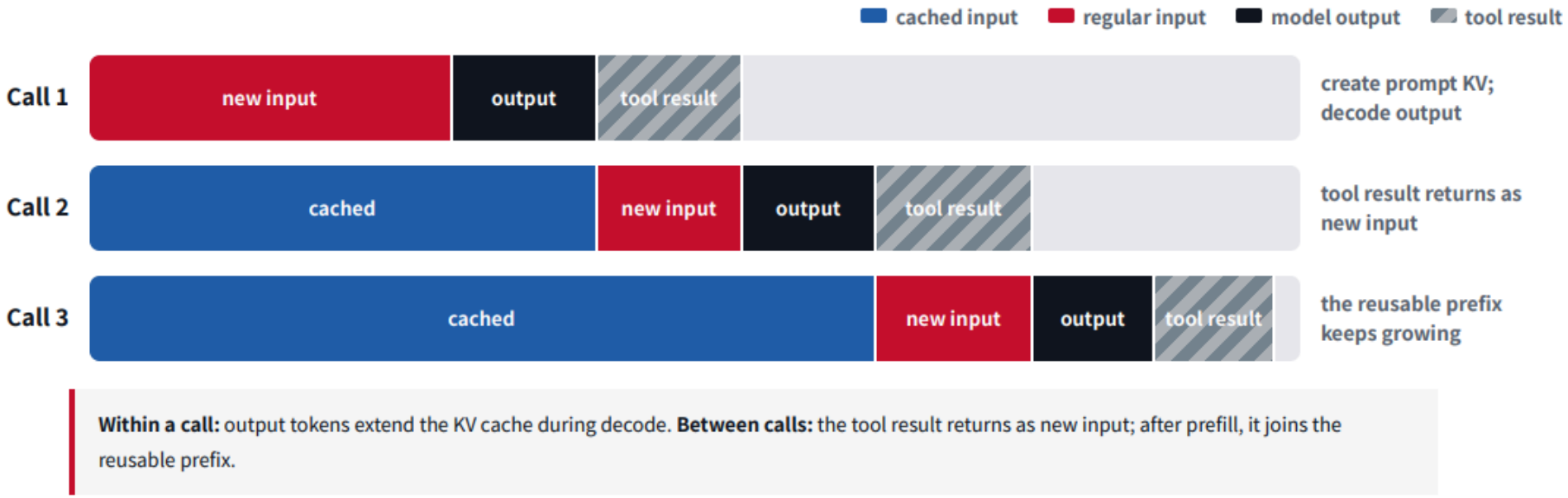
Same model, same answer. Only the compute is reused

ANTHROPIC, OPENAI

Real agent loops read a median 84% of input from cache. Below 80%? Something breaks it

Anthropic, Optimizing for cost and intelligence, 2026

CACHING VISUALISED



Each call reads the cached prefix and computes only the **new** part

WHAT BREAKS THE CACHE

The prefix changes

Everything after the change is computed again

- Edit the system prompt or an old message
- Add or remove tools: MCP server, plugin
- Compact the conversation
- Many images in the history
- Upgrade the harness

A different cache

Same prefix, but no cache for it yet

- Switch the model: each model has its own cache
- Change the effort level (fixed on GPT-6 and Claude 5.5, native API only)

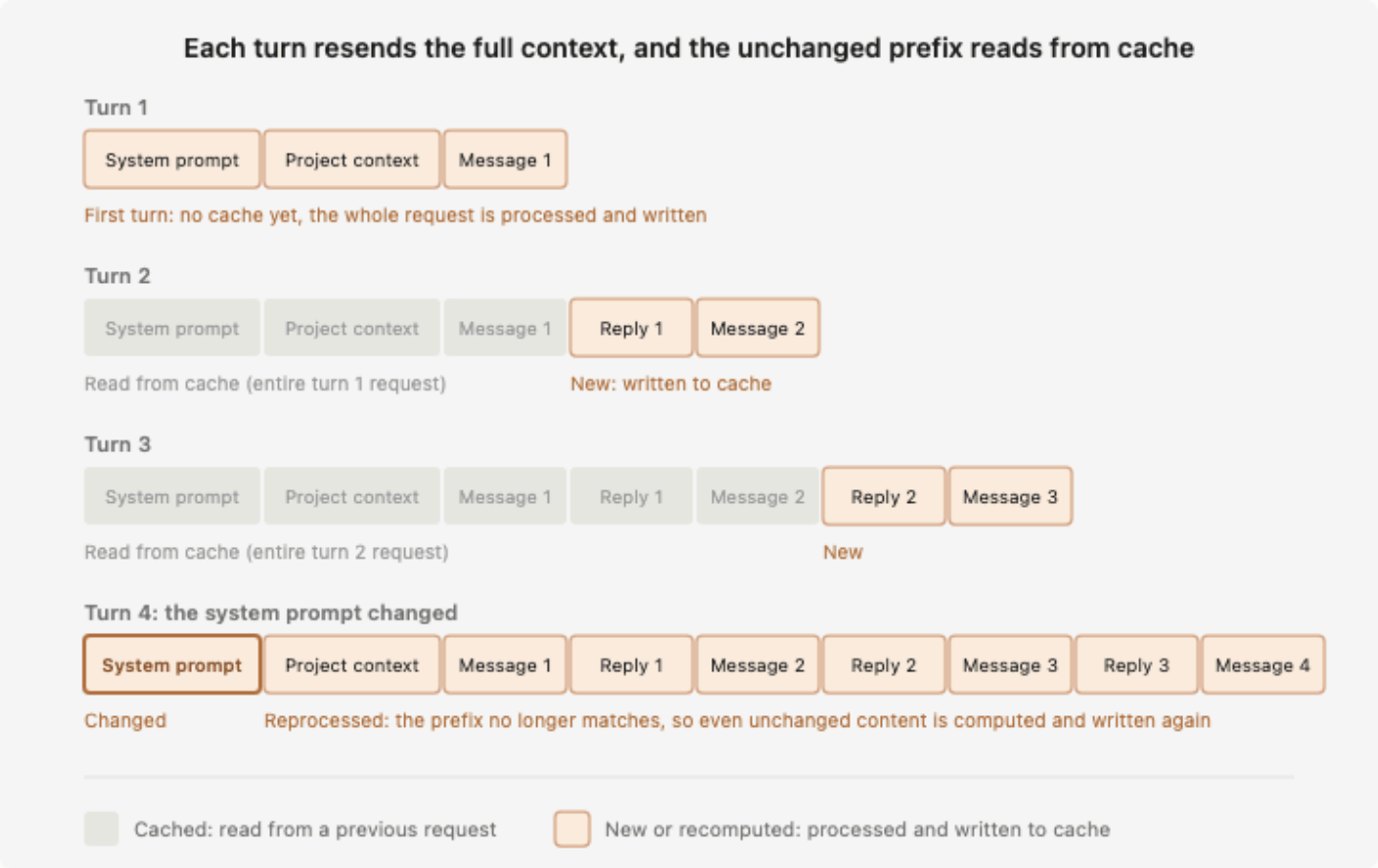
Time runs out

The TTL counts from the start of the request

- Claude: 5 min, or 1 h for 2× the write price
- OpenAI GPT-6: 30 min
- Earlier OpenAI: 5–10 min, up to 24 h extended

One model switch at 220K tokens ≈ 11 cached steps of cost

CACHE MISS: CHANGE THE PREFIX, PAY FOR EVERYTHING AGAIN



A change anywhere **invalidates everything after it**, even content that did not change

THREE WAYS TO CACHE

Implicit

PROVIDER DECIDES

The provider caches common prefixes. You change nothing. Best effort: no hit is guaranteed

OPENAI · GEMINI

Breakpoints

YOU MARK THE PREFIX

A marker on a block: "cache everything up to here". The next request reads the longest entry that still matches. Usually up to 4 markers

CLAUDE · BEDROCK · OPENAI GPT-5.6+

Cache objects

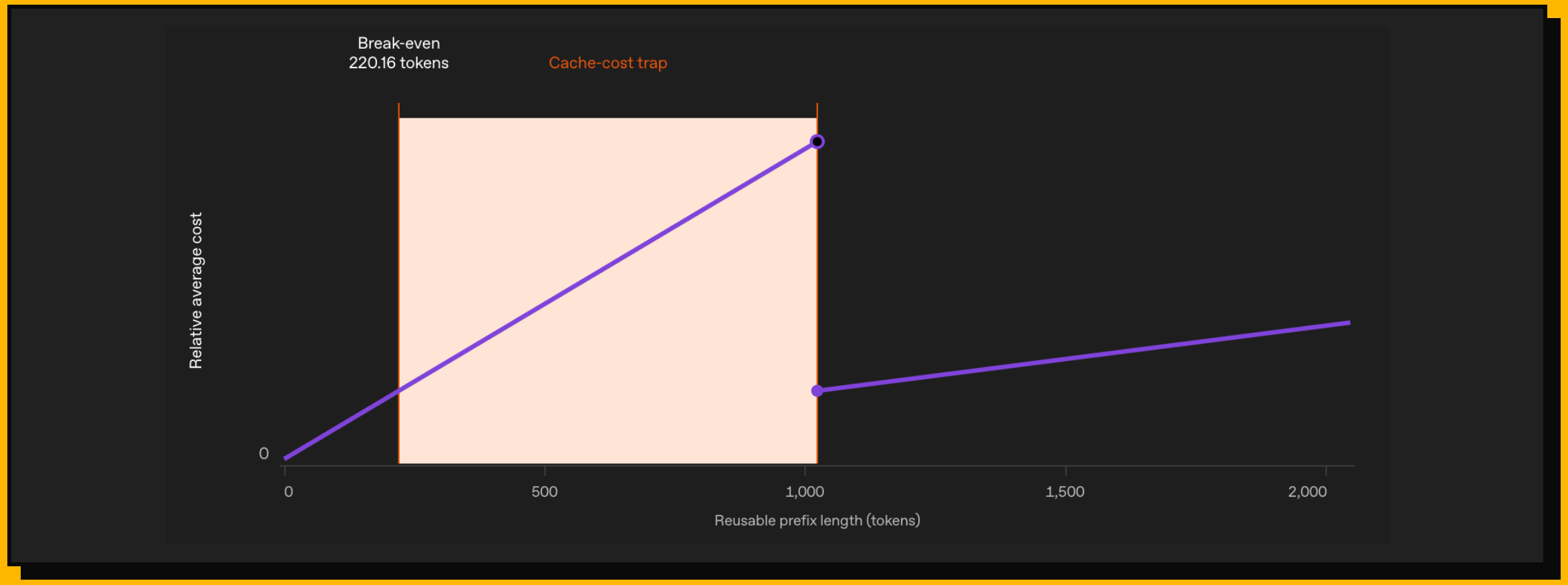
YOU UPLOAD ONCE

Upload the content, get a cache ID, refer to it later. You pay for storage per hour until the TTL ends

GEMINI

Agents use the first two: a growing chat needs a prefix that moves forward

CACHE-COST TRAP: SHORT PREFIXES ARE NOT CACHED



Below 1,024 tokens nothing is cached: a **1,000-token** prefix costs more than a **2,000-token** one

OpenAI, Prompt caching guide · minimum 1,024 tokens · read 0.1× · write 1.25× · 9 reads per write

BEST PRACTICES: APPEND, DON'T EDIT

01 STABILIZE THE PREFIX

- Static first, dynamic last
- Pick model and effort at the start
- No tool changes mid-task

02 APPEND NEW STATE

- New info goes in new messages
- Rewind, don't argue: the old cache still works
- Compact at natural breaks

03 KEEP IT WARM

- 1 h TTL if you pause often
- Pre-warm for low latency
- A 4-min answer leaves ~1 min of a 5-min TTL

04 MEASURE REUSE

- Watch the cache hit rate
- Aim for 80%+
- A drop means something broke the prefix

Rule of thumb: append, don't edit. Pay for a cache break only at a natural break

1.4

CONTEXT ENGINEERING

Not more information. The **smallest set** of high-signal tokens for the next step.

WHAT IS CONTEXT ENGINEERING?

Caching makes long context **cheaper**. Context engineering makes it **smaller**.

> **"Finding the smallest possible set of high-signal tokens that maximize the likelihood of some desired outcome"**
>
> — ANTHROPIC

> **"The delicate art and science of filling the context window with just the right information for the next step"**
>
> — ANDREJ KARPATHY

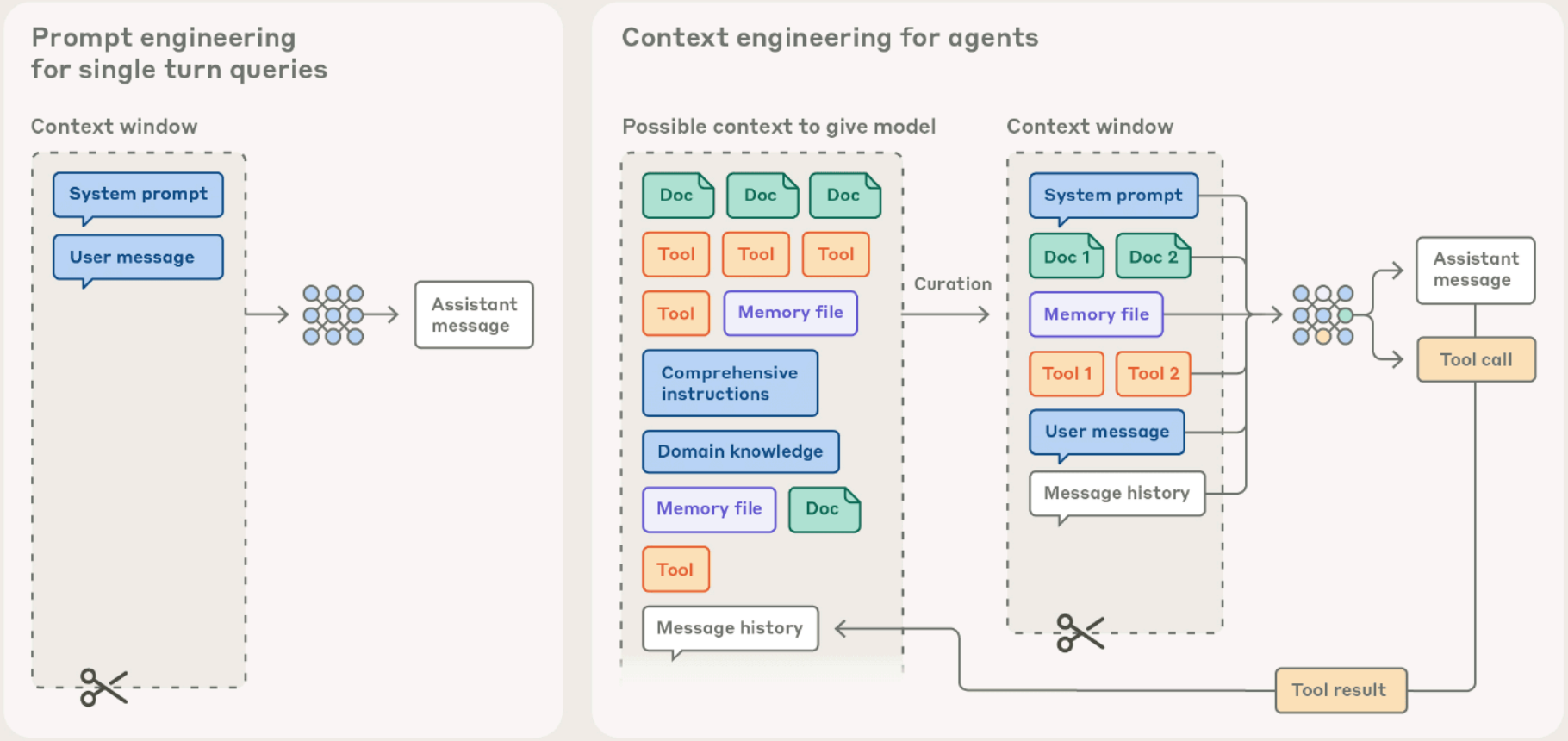
> **"The art of providing all the context for the task to be plausibly solvable by the LLM"**
>
> — TOBI LÜTKE, SHOPIFY

PROMPT ENGINEERING
Tailor the input for one model response

→

CONTEXT ENGINEERING
Build a system that gives the model the right information over many turns

Prompt engineering vs. context engineering



Anthropic, Effective context engineering for AI agents, 2025

FOUR MOVES OF CONTEXT ENGINEERING

// 01

✍️ WRITE

Save it outside the window, so it survives

- Notes and memory files
- Offload raw data to files
- Todo list (recitation)

LOSES
Freshness: memory goes stale

// 02

🎯 SELECT

Pull in only what the next step needs

- Just-in-time retrieval
- Progressive disclosure: skills, tool search, `--help`

LOSES
What the agent doesn't know to ask for

// 03

✂️ COMPRESS

Fewer tokens for the same information

- Token-efficient tools
- Tool result clearing
- Compaction

LOSES
Details, and you can't see which

// 04

📦 ISOLATE

Do heavy work in another window

- Subagents
- Code execution sandbox

LOSES
The parent's context at the handoff

For any new agent feature, ask: which of the four is it, and what does it lose?

LangChain, Context Engineering for Agents, 2025 · Anthropic, Effective context engineering, 2025

// 01 ✍️ **WRITE** Save it outside the window, so it survives

WRITE: SAVE CONTEXT OUTSIDE THE WINDOW

A · TASK MEMORY

LIVES UNTIL THE TASK IS DONE

Problem: the goal gets lost in the middle. After compaction or in a new session, the agent starts with an empty head

- **Offload raw data** - big outputs go to disk, only a path stays. Restorable
- **Plan + recitation** - a todo list, updated each step, keeps the goal at the end of the context
- **Scratchpad** - findings and decisions as it works, e.g. `NOTES.md`
- **Handoff** - progress file + git + task list in JSON. `SPEC.md` for a fresh session

B · LONG-TERM MEMORY

LIVES ACROSS TASKS

Problem: each session forgets what the last one learned

- **Human-written** - `AGENTS.md` / `CLAUDE.md`. Always loaded, so keep it short
- **Agent-written during work** - the agent decides to save a fact. Claude Code auto memory, ChatGPT memory
- **Consolidated offline ("dreaming")** - a background job reads old sessions and merges them. Codex memories

WHAT WRITE COSTS

Read back	Stale	Leaks	Who writes
A note helps only if it is read at the right time. That is a Select question	Wrong memory = poisoning that persists across sessions	Memory can leak into unrelated tasks	Agent: spends attention mid-task. Offline job: free, but arrives late

Manus, 2025 · Anthropic, Effective harnesses for long-running agents, 2025 · LangChain, 2025

WRITE: TWO WAYS TO REMEMBER

📝 CLAUDE CODE AUTO MEMORY

NOTES DURING THE DAY

- **Who writes** - the main agent, during the session, with its file tools
- **At start** - first 200 lines or 25 KB of `MEMORY.md`, the index
- **On demand** - topic files, read with file tools
- **Scope** - per git repo. On by default

🌙 CODEX MEMORIES

A DIARY WRITTEN AT NIGHT

- **Who writes** - a background job: one LLM call per idle session (6 h+), then a sub-agent merges the notes
- **At start** - `memory_summary.md`, cut to 2,500 tokens
- **On demand** - grep `MEMORY.md` and rollout summaries
- **Scope** - global per user. Off by default

KINDS OF MEMORY

Semantic

Facts: "user prefers uv", project structure

Episodic

Past experiences: "last time this test failed because…"

Procedural

How to do things: AGENTS.md rules, skills

Both use the same trick: a small index upfront, details on demand

CoALA, Sumers et al., 2023 · Claude Code memory docs · Codex source

// 02 🎯 **SELECT** Pull in only what the next step needs

SELECT: WHO PICKS THE CONTEXT, AND WHEN

A · UPFRONT

THE HARNESS SELECTS BEFORE THE CALL

- **Always loaded** - system prompt, AGENTS.md, skill metadata, `@file`
- **Classic RAG** - top-k chunks in the prompt. The model can't ask for more
- **Tool loadout** - pick tools per task. Tool choice gets confused above ~30 tools

B · ON DEMAND

THE AGENT SELECTS AT RUNTIME

- **Agentic search** - grep, glob, read ranges. Claude Code: grep beat vector DBs
- **Semantic search as a tool** - Cursor: +12.5% accuracy on top of grep
- **Progressive disclosure** - small index always loaded, details on demand

C · HYBRID

MOST REAL AGENTS

- Claude Code loads CLAUDE.md upfront and greps for the rest
- Upfront fits content that changes less often, e.g. legal or finance

WHAT SELECT COSTS

Slower
On demand = more turns, and each result adds tokens

Unknown unknowns
A weak skill or tool description = never loaded

Noise
Wrong chunks distract. Stale indexes give old code

Cache
Changing loaded tools breaks it. Mask, don't remove

Anthropic, 2025 · Pragmatic Engineer, Building Claude Code, 2026 · Cursor, Semantic search, 2025 · Drew Breunig, 2025 · Manus, 2025

SELECT: TWO AXES, ONE PATTERN

	KEYWORD / STRUCTURE: GREP, PATHS	SEMANTIC: EMBEDDINGS
Upfront HARNESS	**AGENTS.md, @file** Always there, cheap, predictable	**Classic RAG** Embed the query, top-k chunks
On demand AGENT	**Claude Code, Codex** grep / glob / read, refine the query	**Cursor semantic search tool** Same embeddings, the agent decides when

PROGRESSIVE DISCLOSURE: THE SAME PATTERN IN THREE PLACES

SKILLS	name + description	→ SKILL.md body	→ bundled files
TOOLS	tool search stub	→ full definition (−85% tokens)	→ CLI `--help`, MCP as code files
MEMORY	memory_summary.md	→ grep MEMORY.md	→ rollout summaries

// 03 ✂️ **COMPRESS** Fewer tokens for the same information

COMPRESS: WHERE WE CUT

A · AT THE SOURCE

BEFORE TOKENS ENTER THE CONTEXT

- **Token-efficient tools** - pagination, filters, truncation. Claude Code caps tool output at 25K tokens
- **Output filters** - RTK: failures only, short `git status`. 60–90% less bash output
- **Terse model output** - Caveman, "80% of ASD-STE100". DIY, weak evidence

B · DISCARD

IN THE HISTORY, NO LLM

- **Tool result clearing** (observation masking) - replace old outputs with a placeholder, keep the call
- Same for old thinking blocks
- Restorable, if the output was offloaded to a file (Write)

C · CONDENSE

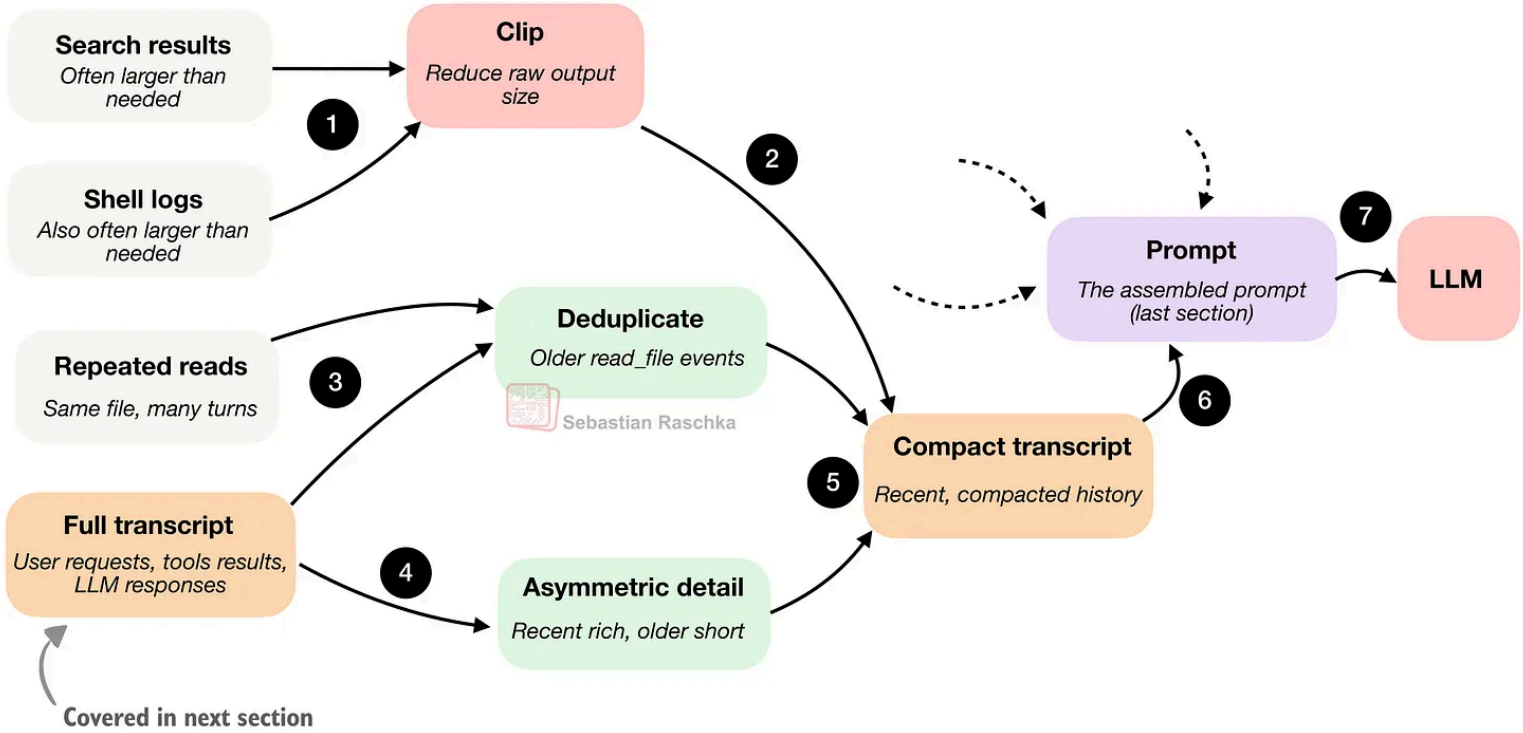
IN THE HISTORY, LLM, LOSSY

- **Compaction** - summarise the history, start a new window with the summary
- **Claude Code** keeps decisions, open bugs, 5 recent files. Steer with `/compact <focus>`
- **Codex** gets an encrypted item from the server. You can't read what it kept

HYBRID **Mask first, summarise only when masking is not enough: −7% cost vs masking, −11% vs summary**

Anthropic, Writing tools for agents, 2025 · RTK · JetBrains, The Complexity Trap, NeurIPS 2025 · Claude Code docs · OpenAI Compaction docs

COMPRESS: A PIPELINE, NOT ONE SUMMARY



Clip big outputs, drop repeated reads, keep **recent detail rich** and older detail short

Sebastian Raschka, Components of a Coding Agent

COMPRESS: SIMPLE BEATS CLEVER

½ cost

Observation masking vs LLM summary. Same solve rate

JETBRAINS, NEURIPS 2025

−84%

Tokens in a 100-turn eval with context editing, +29% on agentic search

ANTHROPIC, 2025

0 LLM calls

Masking is free. Summary costs a call and a buffer in the window

CLAUDE CODE, 2026

NEVER REMOVE

WHAT THE NEXT STEP NEEDS

- **Errors and stack traces** - so the agent doesn't repeat them
- **The plan / todo list** and the user's instructions
- **The last N tool results** - the fresh tail

WHAT COMPRESS COSTS

LOSSY AND CACHE-BREAKING

- **Lost details** - you can't see which. Back only if raw data is on disk
- **New prefix** - after compaction, only system prompt and tools stay cached
- **Clearing breaks the cache** - so clear in big chunks (`clear_at_least`)
- **Summary call** - use a cache-safe fork: same prefix + compaction prompt

Start with masking. Summarise only when you can measure the gain

// 04 📦 **ISOLATE** Do heavy work in another window

ISOLATE: SUBAGENTS

> **"The essence of search is compression": read tens of thousands of tokens, return 1–2K**
>
> — ANTHROPIC, MULTI-AGENT RESEARCH

// 01

🔎 Explorer (read-only)

- Reads a lot, returns a summary
- Claude Code Explore on Haiku
- Lead + subagents beat one Opus by 90%

// 02

🧐 Clean reviewer

- Never saw the build: no bias
- Coder and reviewer share no context

// 03

⚡ Parallel workers

- Independent subtasks at once
- Up to 90% faster research
- Writes stay single-threaded

// 04

🔀 Another model

- A model switch breaks the cache
- Use a subagent with a handoff

DESIGN KNOBS

Context in
Fresh brief (goal, output format, tools, limits) or fork of the parent: shares the cache

Context out
Summary, or files + a reference back. Avoid the "game of telephone"

Permissions
Read-only is the safe default

Anthropic, How we built our multi-agent research system, 2025 · Cognition, Multi-Agents: What's Actually Working, 2026 · Thariq Shihipar, 2026

ISOLATE: CODE EXECUTION, AND THE PRICE

OUTSIDE THE LLM: THE AGENT WRITES A SCRIPT, ONLY THE FINAL OUTPUT ENTERS THE CONTEXT

150K → 2K
Tokens with MCP tools as code files
ANTHROPIC, 2025

−37%
Tokens with programmatic tool calling
ANTHROPIC, 2025

+8%
Cost for strictly sequential calls (τ²-bench). No fan-out, no gain
CLAUDE API DOCS

WHAT ISOLATE COSTS

4× / 15×
Tokens: agents vs chat, multi-agent vs chat
ANTHROPIC, 2025

Handoff
The subagent doesn't see the parent. The parent sees only the summary

Coding
"Fewer truly parallelizable tasks than research"

Overhead
Container start, script writing. Pays off only with fan-out or big results

Isolate when the work is heavy, parallel or valuable. Else stay in one window

CONSTRAINT FOR ALL FOUR: THE CACHE

TECHNIQUE	CACHE EFFECT	CACHE-FRIENDLY WAY
Write: notes, todo	None, appended as new messages	Updates go in messages, not in the system prompt
Select: add/remove tools	Breaks all of it, tools are near the start	Mask, don't remove. Tool search with deferred stubs
Select: on-demand loading	None, results are appended	-
Compress: clearing	Breaks from the first cleared result	Clear in big chunks (`clear_at_least`)
Compress: compaction	New history = new prefix	Cache-safe fork for the summary call
Isolate: subagent	Fresh brief = its own new cache	Fork shares the parent's prefix
Isolate: another model	Each model has its own cache	Subagent with a handoff, not a mid-session switch

Append, don't edit. Pay for a cache break only when it removes many tokens

Manus, 2025 · Claude API, Context editing · Thariq Shihipar, Prompt caching is everything, 2026

TAKEAWAYS

IF YOU KEEP ONLY ONE

The context window is the agent's working memory. Everything in it costs money, time and attention, so put in only what the next step needs

01	**The model sees far more than you typed, on every call**	→ Run `/context` once and look
02	**More context isn't free: worse answers, quadratic cost**	→ 1M is a limit, not a target
03	**Caching is the rule everything else must respect**	→ Same model, same tools. Append, don't edit
04	**Context engineering = smallest set of high-signal tokens**	→ For each token: does the next step need it?
05	**Four moves: Write, Select, Compress, Isolate**	→ For any new feature: which of the four is it?
06	**Simple beats clever: masking, grep, a todo file**	→ Add complexity only when you can measure it
07	**Every technique throws something away**	→ Keep spec, progress and notes on disk

END OF PART 1

Q&A

PART 2

WORKSHOP

Build & compete: **manage context** for trader agents.